



SIMPLE PROGRAMMABLE EVENT TIMER

Microprocessors Project Report

Prepared by

Emre Ekici

Student ID: 64230150

Sude Filiz Diren

Student ID: 64230144

Department of Computer Engineering

Project Features

- 4×4 Matrix Keypad Input
- 16×2 LCD Interface
- Timer1-Based Countdown System
- PWM Audio Generation
- Pirates of the Caribbean Melody Alarm
- Emergency Stop Interrupt Function
- Proteus Simulation and Hardware Implementation

Submission Date

19.06.2026

1. Objective

The objective of this project is to design and implement a microcontroller-based event timer using the PIC16F877A microcontroller. The system allows the user to enter a desired countdown duration through a 4×4 matrix keypad and monitor the remaining time on a 16×2 LCD display. When the countdown reaches zero, an audible alarm melody is generated using the PWM module and a speaker. The system includes an emergency stop mechanism implemented through an external interrupt. This feature immediately halts the countdown and activates a warning siren when the emergency button is pressed. The project aims to demonstrate the practical use of timers, interrupts, keypad interfacing, LCD communication, PWM-based sound generation, and embedded system programming in a real hardware environment.

2. Background Information

Microcontrollers are programmable integrated circuits that combine a central processing unit, memory resources, and peripheral modules within a single device. They are widely used in embedded systems where specific control and monitoring tasks must be performed efficiently. The PIC16F877A is an 8-bit RISC microcontroller that provides Flash program memory, RAM, EEPROM, digital input/output ports, timer modules, interrupt capabilities, and communication interfaces. Due to its versatile architecture and low hardware requirements, it is frequently used in educational, industrial, and automation applications. In this project, the PIC16F877A microcontroller serves as the main control unit of the event timer system.

Accurate time measurement is an essential requirement in many embedded applications. Hardware timer modules are commonly used to generate precise delays and periodic events without continuously occupying the processor. The PIC16F877A includes three timer modules, namely Timer0, Timer1, and Timer2, each designed for different timing purposes. Timer1 is a 16-bit timer that can be configured with selectable prescaler values to achieve accurate timing intervals. By generating periodic interrupts, timer modules allow time-dependent operations to be executed with high precision. In this project, Timer1 is configured to generate regular interrupts that are used to maintain the countdown process.

Interrupts provide a mechanism for responding immediately to internal or external events. When an interrupt occurs, the microcontroller temporarily suspends the main program and executes a dedicated Interrupt Service Routine (ISR). This approach increases system efficiency because the processor does not need to continuously poll input devices. External interrupts are especially useful in applications where rapid response to critical events is required. In this project, an external interrupt is used to detect emergency conditions and immediately stop the timer operation.

User input devices are essential components of embedded systems because they enable interaction between the user and the microcontroller. Matrix keypads are widely used due to their simple structure and efficient use of I/O pins. A matrix keypad is arranged in rows and columns, allowing multiple buttons to be connected using a limited number of microcontroller pins. The keypad is scanned by activating rows sequentially and monitoring the corresponding column signals. In this project, a 4×4 matrix keypad is used to enter the countdown duration and control system functions.

Liquid Crystal Displays (LCDs) are commonly employed in embedded systems to provide visual feedback. Character LCD modules based on the HD44780 controller are particularly popular because they are easy to interface with microcontrollers and require minimal hardware resources. LCDs can operate in either 8-bit or 4-bit communication mode. The 4-bit mode reduces the number of required data lines while maintaining full display functionality. These displays are frequently used to present numerical values, status information, and user instructions. In this project, a 16×2 LCD module is used to display timer settings, countdown information, and warning messages.

Pulse Width Modulation (PWM) is a digital signal generation technique widely used in embedded systems. It is commonly applied in motor speed control, power regulation, LED brightness adjustment, and audio signal generation. PWM signals are characterized by a fixed amplitude and variable duty cycle or frequency. When applied to a speaker or buzzer, PWM signals can generate audible tones with different frequencies corresponding to musical notes. The PIC16F877A contains a Capture/Compare/PWM (CCP) module that supports hardware PWM generation with minimal processor overhead. In this project, the CCP1 module is used to generate musical tones and alarm sounds through a buzzer.

Accurate countdown operation requires a stable time base. In this project, Timer1 was configured to generate an interrupt every 100 ms. The PIC16F877A operates with a 20 MHz crystal oscillator. Since PIC microcontrollers execute one instruction every four clock cycles, the instruction clock frequency becomes:

$$\text{Instruction Clock Frequency} = 20 \text{ MHz} / 4 = 5 \text{ MHz}$$

Therefore, the instruction cycle time is:

$$\text{Instruction Cycle Time} = 1 / 5 \text{ MHz} = 0.2 \mu\text{s}$$

A Timer1 prescaler value of 1:8 was selected. This results in a timer tick period of:

$$\text{Timer Tick Period} = (0.2 \mu\text{s} \times 8) = 1.6 \mu\text{s}$$

To generate a 100 ms interrupt interval, the timer must count:

$$\text{Required Counts} = 100000 \mu\text{s} / 1.6 \mu\text{s} = 62500$$

Since Timer1 is a 16-bit timer with a maximum count value of 65536, the timer registers must be preloaded with:

$$\text{Preload Value} = 65536 - 62500 = 3036$$

The decimal value 3036 corresponds to 0x0BDC in hexadecimal form. Therefore, TMR1H and TMR1L were loaded with 0x0B and 0xDC, respectively. Every Timer1 interrupt represents 100 ms, and ten consecutive interrupts are used to obtain an accurate one-second interval for the countdown process.

The buzzer output was generated using the CCP1 module operating in PWM mode. The output frequency depends on the oscillator frequency, Timer2 prescaler, and the PR2 register value. Different musical notes were produced by changing the PWM frequency according to the required note values. Since PR2 is an 8-bit register, its value must remain between 0 and 255. This limitation was considered while selecting the frequencies used for the alarm melody and emergency siren. Using PWM frequency control, the system was able to generate both the "Pirates of the Caribbean" melody and the emergency warning tones through a single buzzer.

The integration of timer modules, interrupt mechanisms, user input devices, display interfaces, and PWM peripherals represents fundamental concepts in embedded system design. These hardware and software components work together to create responsive and reliable real-time systems that are widely used in industrial control, automation, consumer electronics, and monitoring applications. The event timer developed in this project combines these concepts to implement a practical real-time timing and alert system.

3. Materials and Equipment

The following hardware and software components were used in the design, simulation, and implementation of the event timer system:

Hardware Components:

- PIC16F877A Microcontroller
- PICkit 3 Programmer
- 20 MHz Crystal Oscillator
- 22 pF Capacitors (for crystal oscillator circuit)
- 16×2 Character LCD Display
- 10 kΩ Potentiometer
- 4×4 Matrix Keypad

- Passive Buzzer
- Push Button (Emergency Stop)
- 10 k Ω Resistor
- 220 Ω Resistors
- Breadboard
- Jumper Wires
- 5 V DC Power Supply

Software Tools:

- MPLAB X IDE
- XC8 C Compiler
- Proteus Design Suite
- GitHub

4. Procedure

1. A new project was created in MPLAB X IDE and configured for the PIC16F877A microcontroller using the XC8 compiler and a 20 MHz crystal oscillator.
2. The software structure was developed by implementing LCD control functions, keypad scanning functions, Timer1 interrupt routines, countdown control functions, and PWM control functions for audio generation.
3. Additional functions were written to generate a completion melody and an emergency siren using the CCP1 PWM module. The melody was created by generating different frequencies corresponding to musical notes, allowing the "Pirates of the Caribbean" theme to be played through a buzzer.
4. After completing the software, a simulation circuit was designed in Proteus. The circuit included the PIC16F877A microcontroller, a 16 $\times$ 2 LCD, a 4 $\times$ 4 matrix keypad, a buzzer and an emergency stop button.
5. The LCD was connected to PORTD in 4-bit mode, while the keypad was connected to PORTB and PORTC pins according to the software design.
6. The buzzer was connected to the CCP1 (RC2) PWM output pin through a 220 Ω current-limiting resistor, and the emergency stop button was connected to the RB0 external interrupt pin.

7. A 20 MHz crystal oscillator and two 22 pF capacitors were connected to provide the clock source required by the microcontroller.
8. The generated HEX file was loaded into the Proteus simulation, and the operation of the timer, keypad input, LCD display, PWM-generated melody, and emergency stop function was verified.
9. After successful simulation results were obtained, the hardware components were assembled on a breadboard according to the circuit design.
10. The PIC16F877A microcontroller was placed on the breadboard and connected to the power supply rails.
11. The crystal oscillator, capacitors, reset circuit, LCD module, keypad, buzzer, and emergency stop button were connected to their respective microcontroller pins.
12. The compiled HEX file was programmed into the microcontroller using the PICKit 3 programmer.
13. The completed hardware system was powered using a 5 V supply and tested under different operating conditions.
14. Various countdown values were entered through the keypad, and the countdown operation was monitored on the LCD display.
15. Finally, the alarm melody, PWM audio output, and emergency-stop functionality were tested to verify that the system operated as intended.

5. Results

The event timer system was successfully tested in both the Proteus simulation environment and the physical hardware implementation. The system correctly accepted user input through the keypad, displayed information on the LCD, performed countdown operations using Timer1 interrupts, and generated audio alerts through the PWM module.

No significant timing errors were observed during testing, and the countdown operation remained stable throughout multiple test runs.

5.1 Proteus Simulation Results

The complete circuit was first tested in Proteus before hardware implementation. The simulation verified the correct operation of the keypad interface, LCD display, countdown mechanism, PWM-generated alarm melody, and emergency stop functionality.

Proteus Simulation Video Link: <https://youtu.be/PwnsbgvCfFU>

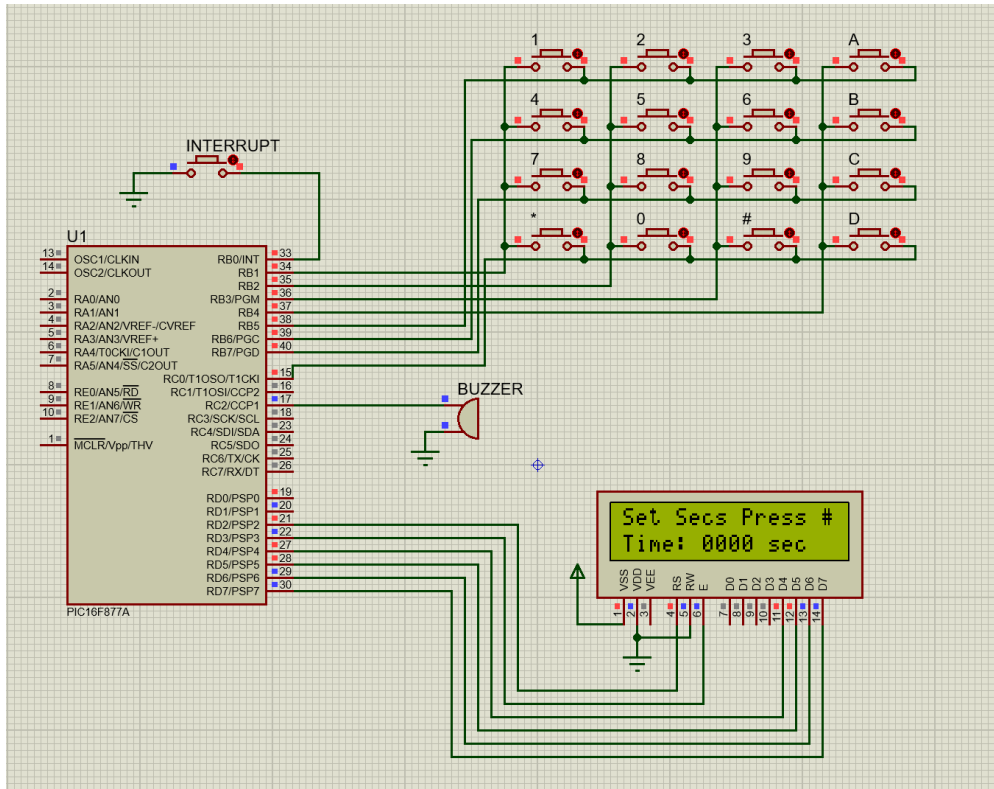


Figure 1: Proteus simulation circuit.

5.2 Hardware Implementation Results

After successful simulation results were obtained in Proteus, the event timer system was assembled on a breadboard using the required hardware components. The PIC16F877A microcontroller was programmed using the PICkit 3 programmer and tested under real operating conditions. The hardware implementation included the PIC16F877A microcontroller, 16×2 LCD display, 4×4 matrix keypad, passive buzzer, emergency stop button, crystal oscillator, and supporting components.

Figure 2 shows the completed hardware implementation assembled on the breadboard.

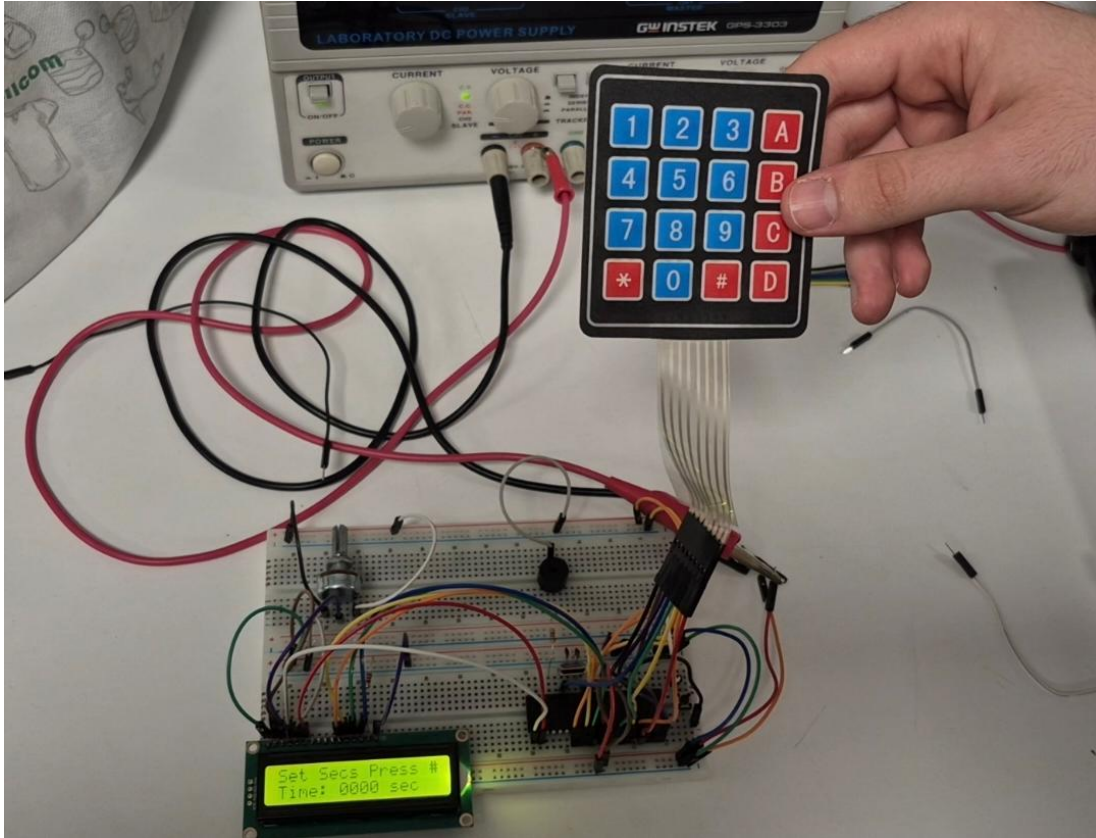


Figure 2: Completed hardware implementation of the event timer system.

The system was tested by entering different countdown values through the keypad. The entered values were successfully displayed on the LCD, and the countdown operation proceeded correctly using Timer1 interrupt-based timing.

Figure 3 shows the LCD display during normal countdown operation.

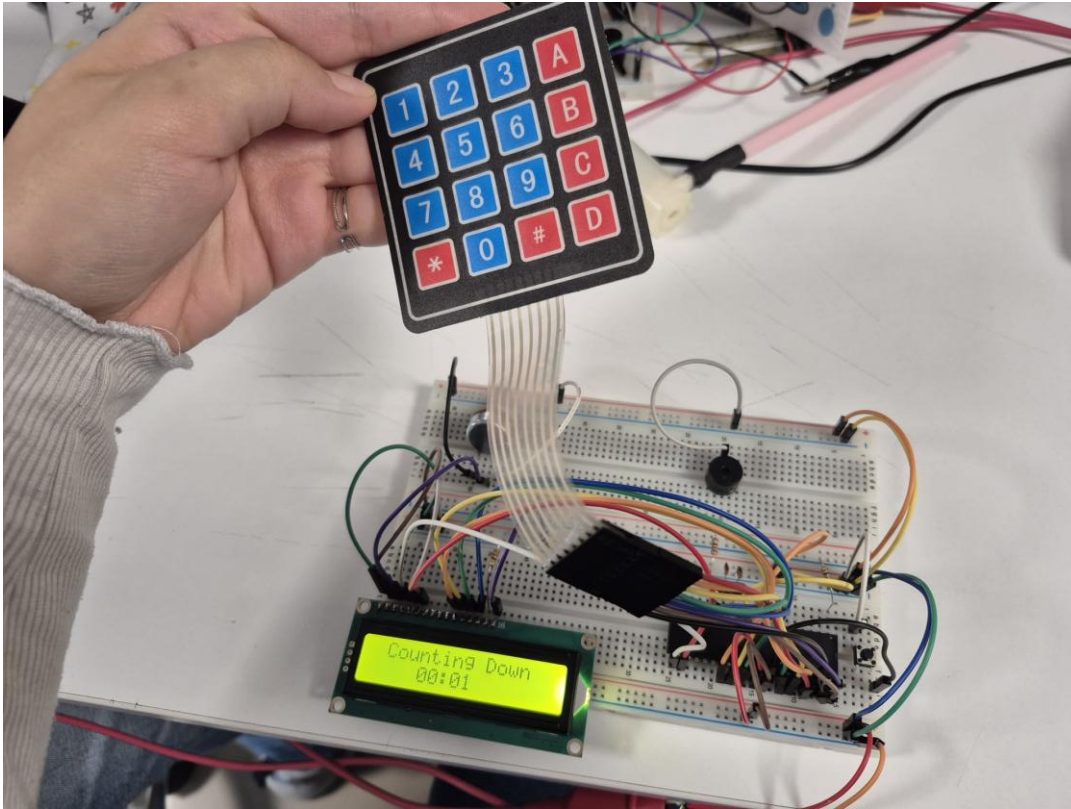


Figure 3: LCD displaying the countdown process.

When the countdown reached zero, the CCP1 PWM module generated the programmed alarm melody. The system successfully played the Pirates of the Caribbean theme through the buzzer, demonstrating the correct operation of the PWM-based audio generation subsystem.

The emergency stop functionality was also tested by pressing the external interrupt button connected to the RB0 pin. When activated, the countdown process immediately stopped and the warning siren was generated through the buzzer. This verified the correct implementation of the interrupt service routine and emergency response mechanism.

Figure 4 demonstrates the emergency stop function during operation.

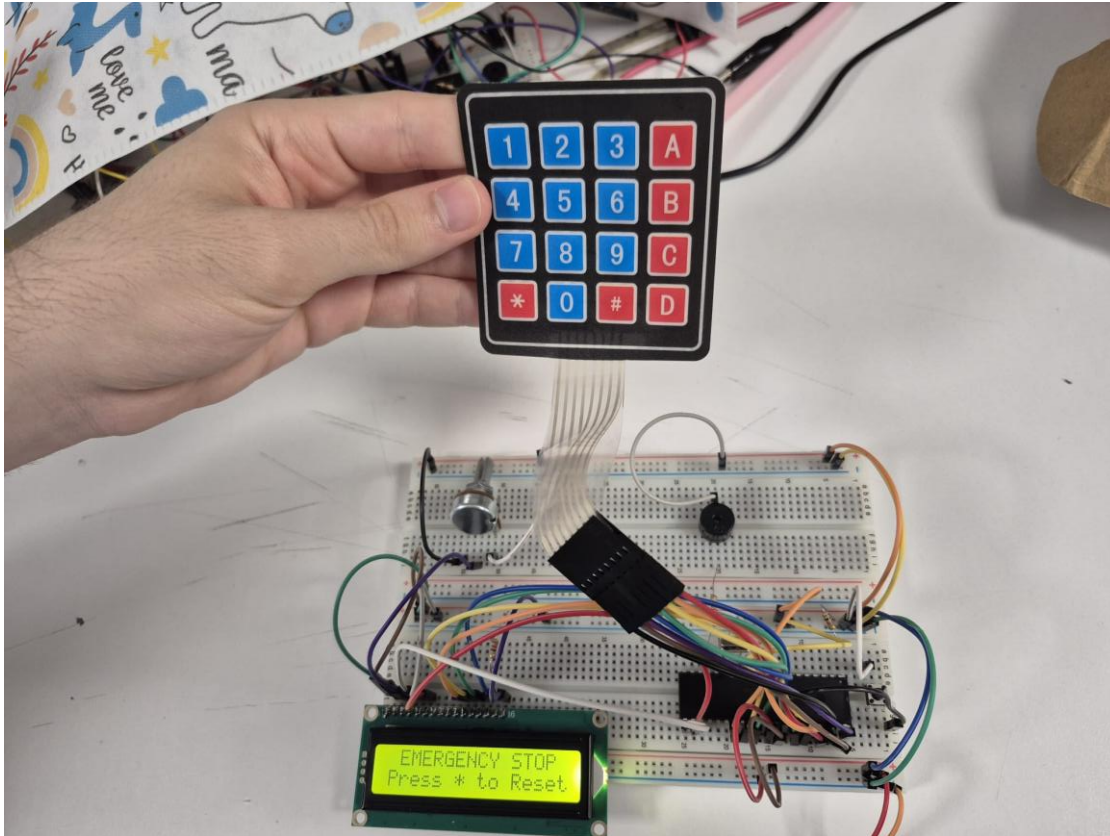


Figure 4: Emergency stop functionality activated through the external interrupt button.

The hardware tests confirmed that the keypad interface, LCD communication, Timer1 interrupt operation, PWM audio generation, and external interrupt handling worked reliably under real operating conditions.

Figure 5 shows the project team together with the completed hardware prototype.

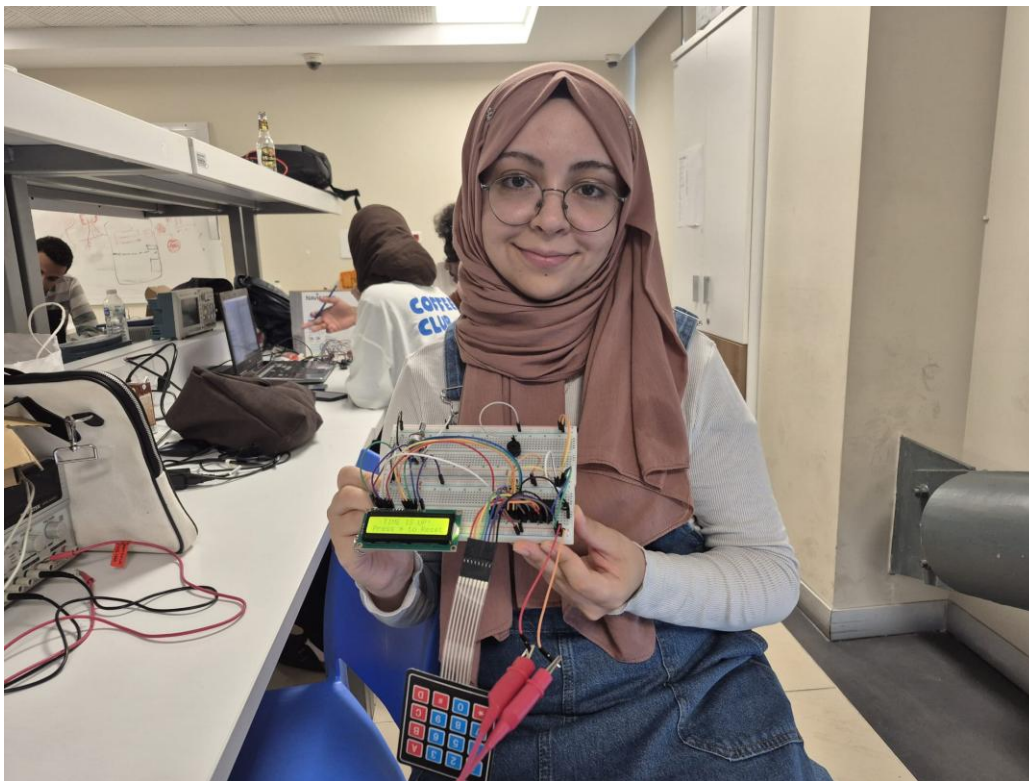
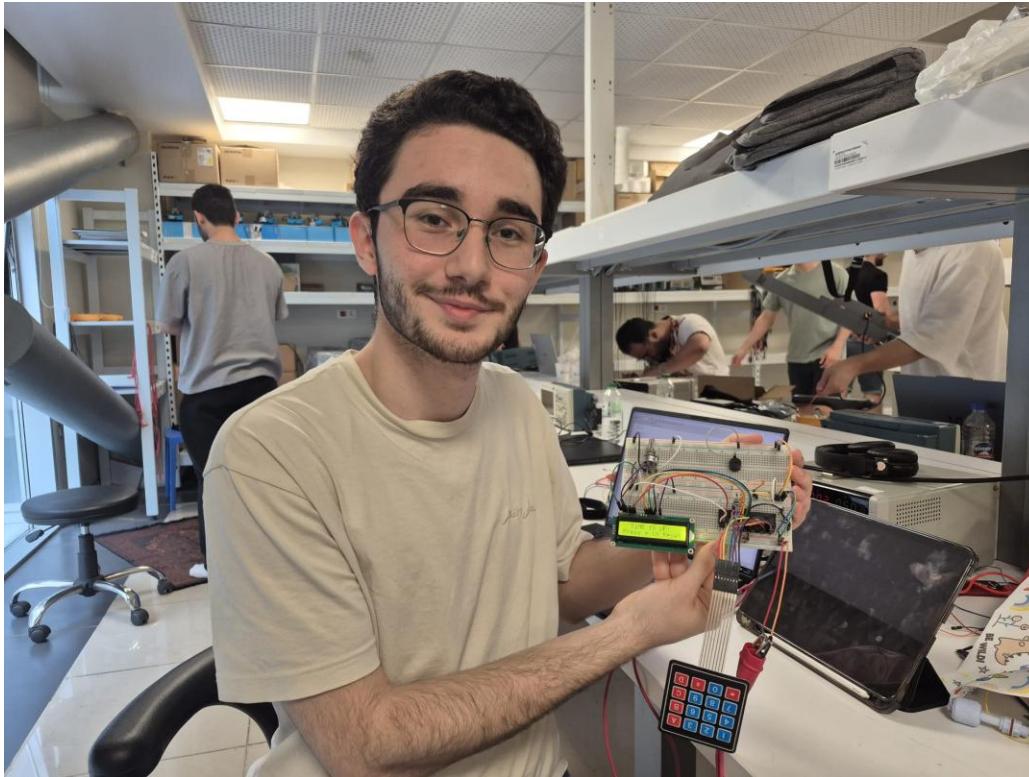


Figure 5: Project team with the completed event timer system.

6. Discussion and Conclusion

Discussion:

The event timer was successfully implemented and tested both in Proteus and on real hardware. The system was able to receive user input through the keypad, display information on the LCD, perform accurate countdown operations using Timer1 interrupts, and generate audio alerts through the PWM module. The emergency stop feature also responded immediately when the external interrupt button was pressed, demonstrating the effectiveness of interrupt-driven event handling.

Timer1 was configured using an appropriate prescaler value to generate periodic interrupts for accurate countdown timing. The interrupt interval was selected based on the 20 MHz crystal frequency, allowing the software to maintain precise one-second countdown updates.

One of the challenges encountered during the hardware implementation was related to the breadboard connections. Some jumper wires were not firmly connected, which caused intermittent operation of certain components. As a result, the circuit was carefully inspected and all connections were checked and corrected. After securing the jumper wires, the system operated reliably.

Another challenge was related to generating audio signals using the CCP1 PWM module. The output frequency depends on the PR2 register value, which must remain within the 8-bit range of the Timer2 hardware. During development, it was necessary to calculate appropriate PR2 values for the desired musical note frequencies while ensuring that the register limits were not exceeded. This issue was resolved by selecting suitable frequency values and implementing limits within the software.

Overall, the project provided practical experience with timers, interrupts, keypad interfacing, LCD communication, PWM-based sound generation, and hardware debugging. It also demonstrated the importance of testing both in simulation and on physical hardware, since some issues only became apparent during real-world implementation.

Conclusion:

In this project, an event timer system based on the PIC16F877A microcontroller was successfully designed, simulated, and implemented on real hardware. The system allows users to enter a countdown duration through a keypad, monitor the remaining time on an LCD display, and receive audible notifications when the countdown is completed.

The project successfully integrated several important embedded-system concepts, including timer modules, interrupt handling, keypad scanning, LCD interfacing, and PWM signal generation. In addition, the PWM module was used to generate musical tones, allowing the system to play the "Pirates of the Caribbean" melody as an alarm signal and a siren sound during emergency conditions.

The results showed that the system operated reliably in both simulation and hardware environments. The project not only achieved its intended functionality but also provided valuable hands-on experience in microcontroller programming, circuit implementation, and troubleshooting. Overall, the event timer serves as an effective demonstration of how multiple hardware and software components can be integrated to create a practical real-time embedded system.

7. References

- [1] Hitachi. (1998). HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver Datasheet.
- [2] Microchip Technology Inc. (2003). PIC16F87XA Data Sheet.
- [3] Microchip Technology Inc. (2024). MPLAB X IDE User's Guide.
- [4] Microchip Technology Inc. (2024). MPLAB XC8 C Compiler User's Guide.
- [5] Verle, M. (2009). PIC Microcontrollers: Programming in C (1st ed.). mikroElektronika.

8. GitHub Repository

A GitHub repository was created to organize and share the project source code, simulation files, documentation, and implementation materials.

Repository:

<https://github.com/sudefilizdiren/PIC16F877A-Event-Timer>